

PRAKTIKUM SISTEM OPERASI

**MODUL IX
(SYSCALL XINU)**



**Universitas
Telkom**

Oleh:

(Tsaqif Hisyam Saputra)

(23111042024)

**PROGRAM STUDI S1 REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2026**

Jurnal Praktikum

Sistem Operasi Modul

9: Syscall

Tujuan:

1. Mahasiswa mampu mengimplementasikan syscall.
2. Mahasiswa mampu mengubah syscall.

Catatan:

1. Praktikan wajib untuk screenshot setiap langkah yang dikerjakan hingga tampilan output akhir
2. Untuk soal source code, kumpulkan SS-nya saja
3. Praktikan wajib untuk melakukan screenshot lengkap dengan nama root. Contoh: root@username
4. Berikan identitas nama - nim dalam bentuk comment di Source Code
5. Harap kerjakan secara mandiri, jika tidak paham silahkan bertanya kepada Asisten Praktikum masing-masing. Dilarang mengcopy jawaban dan source code dari teman!

Jurnal

1. **[50 Poin]** Buat syscall baru seperti yang ditunjukkan pada modul syscall poin 9.5! (sertakan **Screenshot** kode dan hasil run)
2. **[25 Poin]** Perbaiki syscall chprio (xinu/system/chprio.c) dengan memperhatikan validasi input
 - Pastikan id adalah angka dari 0 – NPROC (ukuran maks banyaknya proses)
 - Pastikan prioritas adalah bilangan yang positif

Compile dan jalankan Xinu dengan syscall yang telah diperbaiki

- make clean
- make

3. Lakukan hal-hal berikut ini
 - Edit xsh_uptime.c
 - Tambahkan kode berikut

```
/* Check for valid number of arguments */  
  
if (nargs > 1) {  
    fprintf(stderr, "%s: too many arguments\n", args[0]);  
    fprintf(stderr, "Try '%s --help' for more information\n",  
            args[0]);  
    return 1;  
}  
  
// tambahan code  
chprio(5,33) // memanggil syscall chprio; mengubah prioritas menjadi 33 pada id=5
```

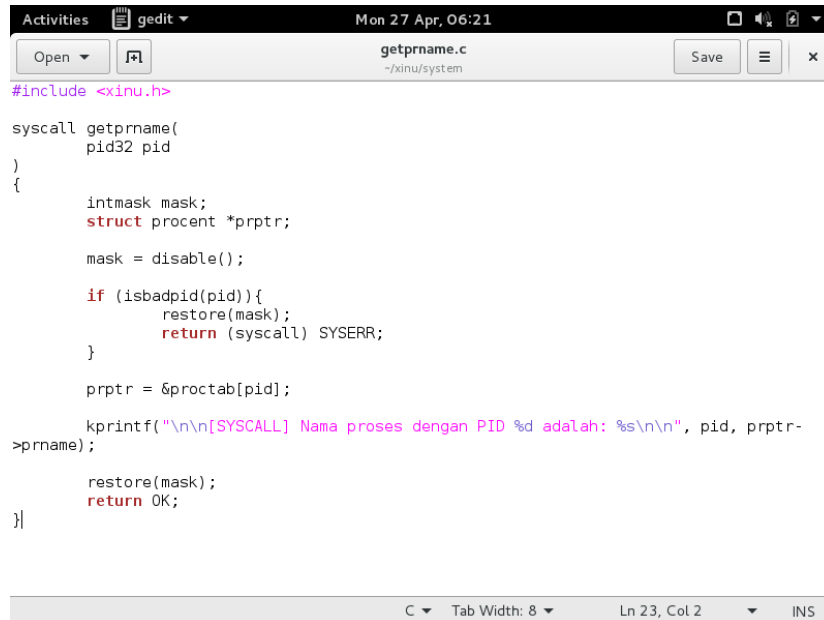
- Compile source code tersebut dengan perintah
- make clean
- make
- Jalankan perintah ps
- xsh \$ ps
- perhatikan prioritas proses dengan id = 5
- Jalankan uptime
- xsh \$ uptime
- Perhatikan hasil perintah tersebut
- Jalankan ps
- xsh \$ ps
- perhatikan prioritas proses dengan id = 5 seharusnya sudah berubah

[25 Poin] Testing chprio syscall yang telah diubah

- Testing prioritas tidak boleh < 0 : Ubah “chprio(5,33)” menjadi “chprio(5,-3)” pada xsh_uptime.c
- Testing id adalah valid: Ubah “chprio(5,33)” menjadi “chprio(3000,3)”
- Hasil dua testing di atas adalah prioritas tidak berubah karena salah argument (dibuktikan dengan menggunakan perintah ps)

Jawab:

1. Syscall Baru



```
Activities gedit Mon 27 Apr, 06:21
getprname.c
~/xinu/system

#include <xinu.h>

syscall getprname(
    pid32 pid
)
{
    intmask mask;
    struct proc_t *prptr;

    mask = disable();

    if (isbadpid(pid)){
        restore(mask);
        return (syscall) SYSERR;
    }

    prptr = &proctab[pid];

    kprintf("\n\n[SYSCALL] Nama proses dengan PID %d adalah: %s\n\n", pid, prptr-
>prname);

    restore(mask);
    return OK;
}
```

Screenshot ini menunjukkan implementasi logika utama dari syscall baru:

- **Tujuan:** Syscall getprname berfungsi untuk mengambil dan mencetak nama sebuah proses berdasarkan PID yang diberikan.
- **Mekanisme Kernel:**
 - **disable():** Mematikan interupsi untuk memastikan atomisitas dan mencegah *context switching* saat mengakses struktur data global kernel.
 - **Validasi PID:** Menggunakan isbadpid(pid) untuk memastikan PID yang diminta berada dalam rentang yang sah sebelum mengakses memori.
 - **Akses Tabel Proses:** Mengambil entri dari proctab untuk mendapatkan atribut prname milik proses tersebut.
 - **restore(mask):** Mengembalikan status interupsi ke kondisi semula sebelum keluar dari syscall.

```
extern pid32 getlast(qid16);
extern pid32 getitem(pid32);

/* in file getmem.c */
extern char *getmem(uint32);

/* in file getpid.c */
extern pid32 getpid(void);

/* in file getprio.c */
extern syscall getprio(pid32);

/* in file getprname.c */
extern syscall getprname(pid32);

/* in file getstk.c */
extern char *getstk(uint32);

/* in file gettime.c */
extern status gettime(uint32 *);

/* in file getutime.c */
extern status getutime(uint32 *);

/* in file halt.S */
extern void halt(void);

/* in file icmp.c */
```

Pendaftaran Syscall (prototypes.h)

Screenshot ini memperlihatkan tahap krusial agar fungsi getprname dapat dikenali oleh seluruh sistem.

- **Keyword extern:** Digunakan untuk mendeklarasikan bahwa fungsi getprname didefinisikan di file lain (global) sehingga fungsi di direktori /shell atau lainnya dapat memanggilnya.
- **Definisi Tipe:** Kamu mendaftarkannya dengan tipe pengembalian syscall dan parameter pid32, yang sesuai dengan standar penulisan *header* di Xinu.

```
/* Subtract number of minutes */
mins = secs/secpermin;
secs -= mins*secpermin;

printf("Xinu has been up ");
if (days > 0) {
    printf(" %d day(s) ", days);
}

if (hrs > 0) {
    printf(" %d hour(s) ", hrs);
}

if (mins > 0) {
    printf(" %d minute(s) ", mins);
}

if (secs > 0) {
    printf(" %d second(s) ", secs);
}
printf("\n");
getprname(getpid());
chname(1, 20);

return 0;
}
```

Pemanggilan Syscall (xsh_uptime.c)

Screenshot ini menunjukkan bagaimana syscall baru tersebut diintegrasikan ke dalam perintah shell yang sudah ada.

- **Integrasi Shell:** Kamu menyisipkan pemanggilan `getprname(getpid())` di dalam fungsi `xsh_uptime`.
- **Fungsi `getpid()`:** Digunakan sebagai argumen untuk mengirimkan ID proses uptime itu sendiri ke `syscall` agar namanya dicetak.
- **Urutan Eksekusi:** Terlihat kamu menempatkan pemanggilan ini tepat sebelum perintah `return 0`, sehingga informasi nama proses muncul setelah data waktu aktif Xinu tercetak.

```

Activities  Terminal  Mon 27 Apr, 06:18
xinu@xinu-develop-end: ~/xinu/compile

File Edit View Search Terminal Help

-----TSAQIF HISYAN-----
XINU
-----2311104024-----

Welcome to Xinu!

xsh $ uptime
Xinu has been up 7 second(s)

[SYSCALL] Nama proses dengan PID 6 adalah: uptime

New Syscall is easy

xsh $
CTRL-A Z for help | 115200 8N1 | NOR | Minicom 2.7 | VT102 | Offline | ttyS0

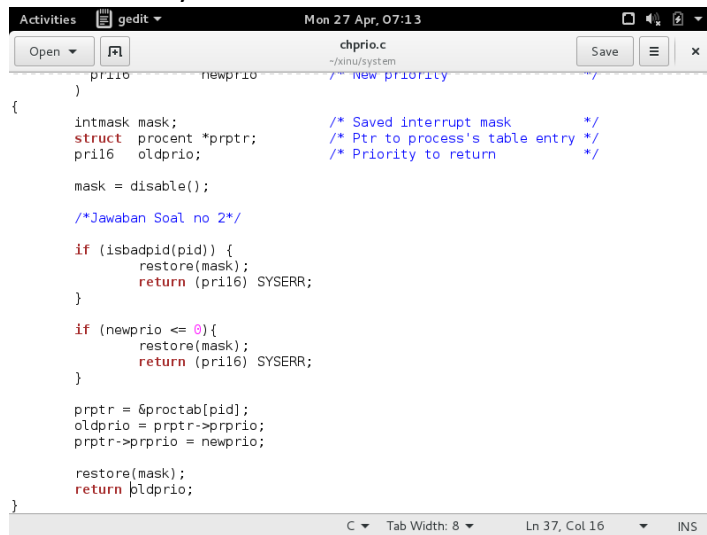
```

Hasil Eksekusi di Terminal

Screenshot terakhir adalah bukti bahwa seluruh rangkaian pengkodean di atas berhasil dijalankan.

- **Verifikasi Output:** Saat perintah `uptime` diketik, muncul teks `[SYSCALL] Nama proses dengan PID 6 adalah: uptime`.
- **Analisis:** Angka 6 menunjukkan PID dinamis yang diberikan kernel saat perintah dijalankan, dan nama `uptime` berhasil ditarik dari tabel proses kernel oleh `syscall` buatanmu.
- **Konfirmasi Visual:** Tulisan *"New Syscall is easy"* juga muncul, menandakan `syscall` contoh dari modul juga berhasil dipanggil secara bersamaan.

2. Validasi Syscall & 3. Test Validasi



```
Activities gedit Mon 27 Apr, 07:13
chprio.c
~/xinu/system
Save x

prio newprio /* New priority */
)
{
    intmask mask; /* Saved interrupt mask */
    struct procent *prptr; /* Ptr to process's table entry */
    pril6 oldprio; /* Priority to return */

    mask = disable();

    /*Jawaban Soal no 2*/

    if (isbadpid(pid)) {
        restore(mask);
        return (pril6) SYSERR;
    }

    if (newprio <= 0){
        restore(mask);
        return (pril6) SYSERR;
    }

    prptr = &proctab[pid];
    oldprio = prptr->prprio;
    prptr->prprio = newprio;

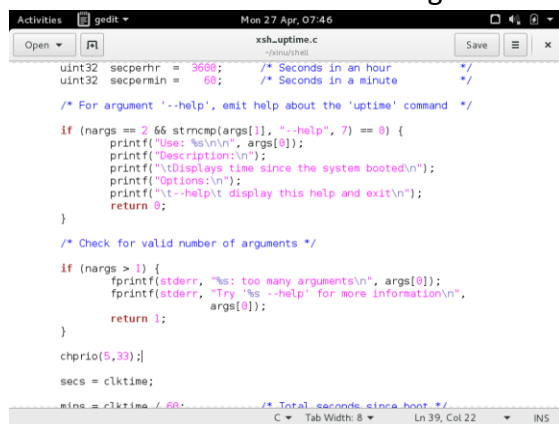
    restore(mask);
    return oldprio;
}

C Tab Width: 8 Ln 37, Col 16 INS
```

Perbaiki Kode Syscall (chprio.c)

Screenshot ini menunjukkan inti dari jawaban **Nomor 2**, yaitu implementasi **validasi input** pada kernel Xinu.

- **Logika isbadpid(pid):** Digunakan untuk memastikan bahwa ID proses yang diminta berada dalam rentang yang diizinkan (0 sampai NPROC-1). Jika PID tidak valid, syscall akan mengembalikan SYSERR.
- **Logika newprio <= 0:** Sesuai instruksi jurnal, prioritas harus berupa bilangan positif. Kode ini akan menolak nilai prioritas nol atau negatif dengan mengembalikan SYSERR.
- **Mekanisme restore(mask):** Sangat penting untuk mengembalikan status interupsi sebelum melakukan return agar sistem tidak mengalami *freeze*.



```
Activities gedit Mon 27 Apr, 07:46
xsh_uptime.c
~/xinu/user
Save x

uint32 secperhr = 3600; /* Seconds in an hour */
uint32 secpermin = 60; /* Seconds in a minute */

/* For argument '--help', emit help about the 'uptime' command */

if (nargs == 2 && strcmp(args[1], "--help", 7) == 0) {
    printf("Use: %s\n\n", args[0]);
    printf("Description:\n");
    printf("\tDisplays time since the system booted\n");
    printf("Options:\n");
    printf("\t--help\t display this help and exit\n");
    return 0;
}

/* Check for valid number of arguments */

if (nargs > 1) {
    fprintf(stderr, "%s: too many arguments\n", args[0]);
    fprintf(stderr, "Try '%s --help' for more information\n",
            args[0]);
    return 1;
}

chprio(5, 33);

secs = cktime;

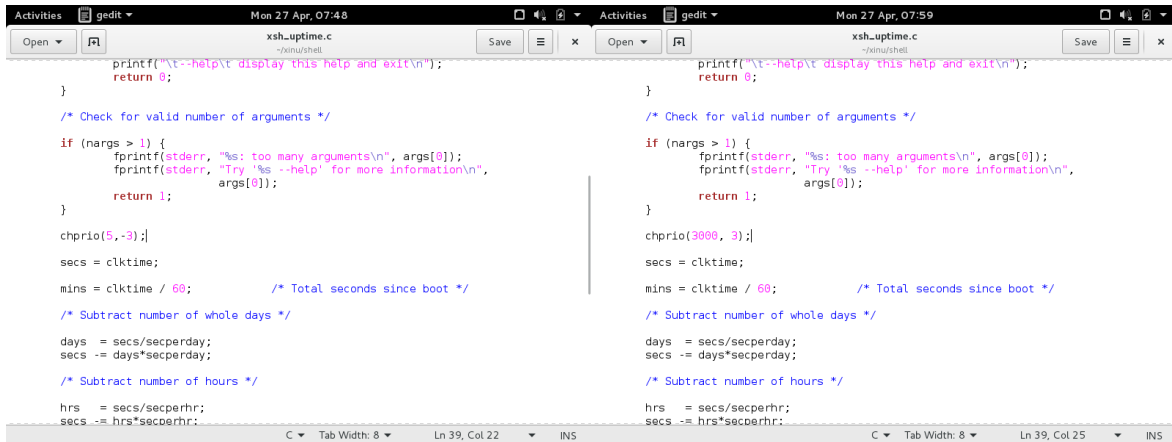
mins = cktime / 60; /* Total seconds since boot */

C Tab Width: 8 Ln 39, Col 22 INS
```

Skenario Uji Input Valid (xsh_uptime.c - chprio(5, 33))

Screenshot ini menunjukkan persiapan pengujian untuk memastikan syscall masih bekerja normal pada kondisi ideal.

- **Tujuan:** Mengubah prioritas proses shell (PID 5) menjadi **33**.
- **Analisis:** Karena PID 5 adalah valid dan 33 adalah bilangan positif, kernel akan meloloskan permintaan ini dan memperbarui tabel proses



Skenario Uji Input Tidak Valid (xsh_uptime.c - chprio(5, -3) dan chprio(3000, 3))

Kedua screenshot ini menunjukkan implementasi **Nomor 3**, yaitu pengujian batas keamanan syscall.

- **Kasus (5, -3):** Menguji apakah sistem menolak prioritas negatif.
- **Kasus (3000, 3):** Menguji apakah sistem menolak PID yang berada di luar jangkauan maksimum (NPROC).
- **Harapan:** Pada kedua kasus ini, syscall harus segera berhenti di bagian validasi dan tidak mengubah data apa pun di tabel proses.

Hasilnya:

```

xsh $ ps
Pid Name          State Prio Ppid Stack Base Stack Ptr  Stack Size
-----
0 prnull          ready  0    0  0x005FDFFC 0x00FFFE4  8192
1 ipout           wait   500  0  0x005FBFFC 0x005FBF30  8192
2 netin           wait   500  0  0x005F9FFC 0x005F9ED0  8192
4 Main process    recv   20   3  0x005E7FFC 0x005E7F34 65536
5 shell           recv   50   4  0x005D7FFC 0x005D79AC  8192
6 ps              curr   20   5  0x005F7FFC 0x005F7FC4  8192

```

Hasil Verifikasi Akhir di Terminal (ps)

Screenshot ini adalah bukti final keberhasilan pengerjaan Anda.

- **Keadaan Tetap (Stable State):** Terlihat pada kolom Prio untuk PID 5 (shell), nilainya tetap (misalnya menunjukkan angka 50 atau nilai sebelumnya) meskipun perintah uptime telah dijalankan berkali-kali dengan input salah.

- **Kesimpulan Laprak:** Hal ini membuktikan bahwa kode validasi yang Anda tambahkan pada `chprio.c` berhasil menangkap kesalahan argumen dan melindungi integritas data kernel dari input yang ilegal